

Take-Home Assignment: Payment Gateway Fraud Detection

Goal

Build a machine-learning model that predicts whether a payment transaction is fraudulent.

You are given a small sample CSV file named `fraud_detection_sample_transactions.csv`. The file contains payment-gateway transaction records and a target column:

- `fraud_label = 1` means fraud
- `fraud_label = 0` means genuine transaction

The sample is intentionally small and only demonstrates the schema. You may use it to understand the columns, create additional synthetic records, or use another public fraud dataset if needed. Clearly state which data you used.

Sample Data Columns

Column	Meaning
<code>request_id</code>	Unique transaction ID
<code>request_time</code>	Transaction time
<code>service_type</code>	Payment method, such as UPI, wallet, IMPS, or netbanking
<code>device_id</code>	Device identifier
<code>merchant_id</code>	Merchant identifier
<code>merchant_state,merchant_city</code>	Merchant location
<code>merchant_type</code>	Merchant risk tier
<code>mcc_code,mcc_title</code>	Merchant category code and category name
<code>issuer_bank</code>	Issuer bank
<code>currency_code</code>	Transaction currency
<code>amount</code>	Transaction amount
<code>request_status</code>	Transaction status
<code>fraud_label</code>	Target: 1 = fraud, 0 = genuine

Your Tasks

1. Understand the data

- Perform a short EDA.
- Report the fraud rate.
- Identify useful fraud signals from the available data, such as unusual amount, high-risk merchant type/MCC, late-night transaction, repeated device activity, payment method, and transaction status.
- Identify fields that should not be used directly because they are unique IDs, PII, or could cause leakage. Explain what derived features could still be created from them.

2. Build a model

- Create at least one baseline model, for example Logistic Regression.
- Create one tree-based model, for example Random Forest, XGBoost, or CatBoost.
- Create useful features from `request_time`, `amount`, and device/merchant behavior where possible.
- Handle categorical variables and missing values.
- Use a time-based train/test split if transaction timestamps are available.
- Handle class imbalance appropriately and explain your choice.

3. Test the model

Evaluate your final model using:

- Precision
- Recall
- F1-score
- PR-AUC or ROC-AUC
- Confusion matrix

Explain which metric is most important for fraud detection and why. Do not rely only on accuracy.

4. Explain production checks

Briefly describe the checks you would perform before releasing the model:

- Input schema and data-type validation.
- Missing-value and feature-range checks.
- Training-serving feature consistency.
- Model version, evaluation result, and rollback plan.

5. Drift detection and monitoring

Assume the model was trained on January–June transactions and receives live July–September transactions.

Describe and demonstrate how you would monitor:

- **Data drift:** compare the training and latest data distributions for at least two features, such as `amount`, `service_type`, `merchant_type`, or `mcc_code`. Use PSI, KS test, or another suitable method.
- **Prediction drift:** compare the fraud-score distribution between the training/validation period and the latest production period.
- **Performance drift:** once new fraud labels arrive, compare current precision, recall, F1, and PR-AUC/ROC-AUC with the original baseline.
- **Business metrics:** monitor fraud loss amount, false declines, approval rate, and manual-review queue size.

State clear actions for each issue: investigate data quality, adjust a decision threshold, train a challenger model, run shadow testing, retrain, or roll back.

Submission

Submit a public GitHub repository containing:

- Your code or notebook.
- The sample data and/or a clearly documented data source.
- A `README.md` with setup/run instructions.
- A short explanation of model choice, testing results, drift monitoring, limitations, and next steps.

Time Box

3–4 hours. Use of AI tools is allowed. We are evaluating your reasoning, model-testing discipline, and practical understanding of monitoring and drift detection.